

UNIDAD 8:

ENTRENAMIENTO SUPERVISADO

CURSO

IA

```
01 import openai
02 from langchain import
03     ChatOpenAI
04
05 def generar_respuesta(prompt):
06     modelo = ChatOpenAI(
07         model = "gpt-4o"
08         temperature = 0.7
09     )
10
11     respuesta = modelo.invoke(
12         prompt
13     )
14     return respuesta.content
15
16 if __name__ == "__main__":
17     prompt = "Explicame qué
18     es la inteligencia artificial"
19     print(generar_respuesta(prompt))
```



Docente:
Gabriel Mosso



BRAINSTORM

Rep.Argentina 786, Sunchales, Sta fe.



GabrielNMosso@GMail.com



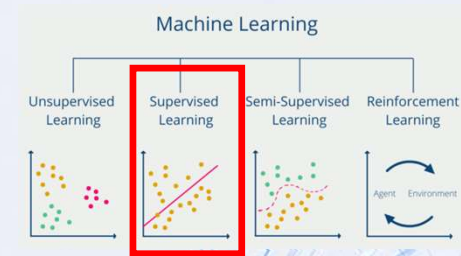
Tel: +54 351 3733383



ENTRENAMIENTO SUPERVISADO

El **entrenamiento supervisado** es una de las formas más comunes y fundamentales de enseñar a una inteligencia artificial.

Se basa en el uso de **datos etiquetados** o sea cada ejemplo que se le muestra al modelo ya viene con la **respuesta correcta**.



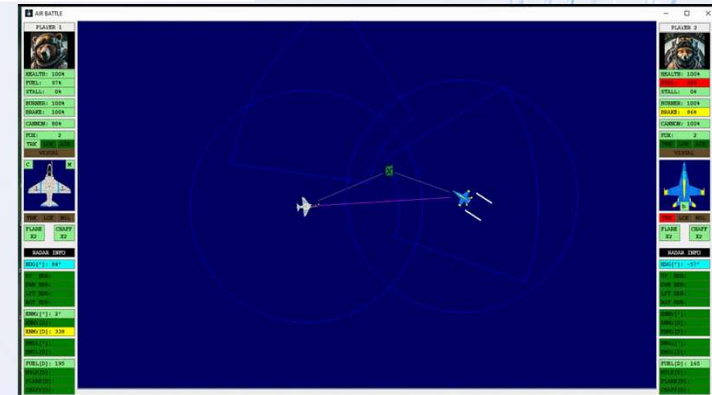
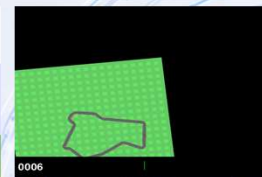
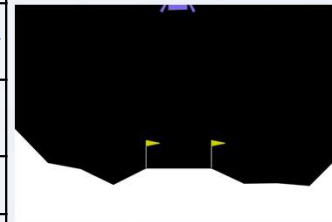
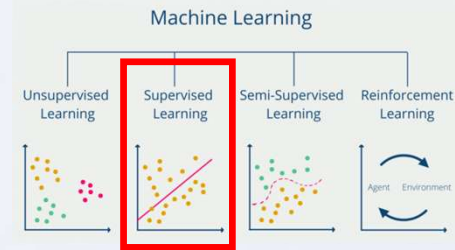
Aspecto	Descripción
Definición	Método de entrenamiento basado en datos etiquetados, donde cada entrada tiene una salida conocida.
Funcionamiento	1. Se entregan ejemplos con entradas y salidas correctas. 2. El modelo aprende a relacionar entradas con salidas. 3. Se evalúa con datos nuevos.
Usos principales	- Clasificación (asignar categorías). - Regresión (predecir valores numéricos).
Ejemplos	- Detectar spam en correos electrónicos. - Predecir el precio de una casa.- Reconocimiento de imágenes.
Requisitos	- Gran cantidad de datos etiquetados. - Etiquetas consistentes y precisas. - Datos representativos del problema.
Ventajas	- Buen rendimiento si los datos son adecuados. - Evaluación objetiva. - Ideal para problemas concretos con resultados esperados.
Desventajas	- Costoso en tiempo y recursos por la necesidad de etiquetado. - Poco flexible con datos no etiquetados. - Riesgo de sobreajuste si no se controla bien.





ENTRENAMIENTO SUPERVISADO: GIMNASIOS DISPONIBLES

Nombre	Plataforma	Tipo de entornos	Ventajas destacadas	URL oficial / Repositorio
Gymnasium (ex OpenAI Gym)	Python	Juegos simples, control clásico, Atari	Modular, ligero, comunidad activa, fácil de integrar con algoritmos estándar	https://gymnasium.farama.org
DeepMind Control Suite	Python + MuJoCo	Control motor, robótica	Física realista, útil para benchmarking	https://github.com/deepmind/dm_control
Unity ML-Agents	Unity + Python	Juegos 3D, simulaciones complejas	Gráficos 3D, multijugador, integración con Deep Learning	https://unity.com/products/machine-learning-agents
PyBullet	Python	Simulación robótica	Gratuito, física precisa, alternativa libre a MuJoCo	https://pybullet.org/ https://github.com/bulletphysics/bullet3
MuJoCo	Python	Simulación avanzada de robots	Alta fidelidad, usado en investigación avanzada	https://mujoco.org
PettingZoo	Python	Multiagente (juegos y simulaciones)	Diseñado para múltiples agentes, integración con Gym y Stable-Baselines	https://www.pettingzoo.ml
Meta-World	Python + MuJoCo	Manipulación robótica multitarea	Reutilización de conocimiento entre tareas, entorno realista de manipulación	https://github.com/rlworkgroup/metaworld
MineRL	Python + Java	Minecraft	Ambiente naturalista, tareas jerárquicas, integración con aprendizaje profundo	https://minerl.io/ https://github.com/minerllabs/minerl
BrainStorm Air Battle	Python + Java	Juego 2D Combate	Desarrollo de estrategias, combate, supervivencia.	

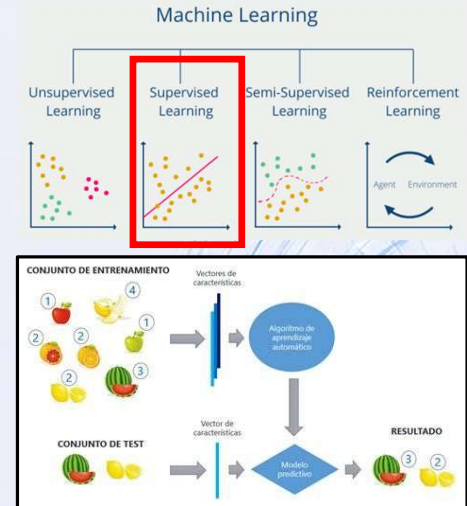




ENTRENAMIENTO SUPERVISADO [IMPLEMENTACION HIBRIDA]

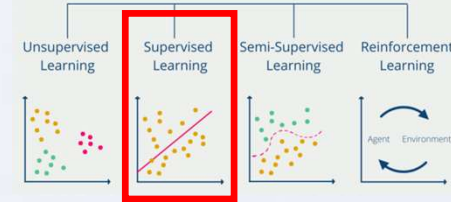
Etapas Inicial	Objetivo	Tareas específicas	Herramientas / Técnicas comunes	Errores frecuentes
1. Crear dataset manual	Generar un dataset inicial con ejemplos correctos de juego	Jugar partidas manualmente y registrar los estados (X) y comandos (Y)	CSV, pandas, grabación de partidas	Registrar casos incorrectos; Datos desbalanceados; pocas partidas para generalizar
2. Entrenar modelo inicial	Que la DNN aprenda a imitar el comportamiento humano	Definir arquitectura de la DNN, compilar, usar .fit() con dataset	TensorFlow/Keras, .fit(), pérdida binary_crossentropy	Overfitting por pocas partidas; aprender patrones erróneos si los datos son malos

Etapas Cíclicas	Objetivo	Tareas específicas	Herramientas / Técnicas comunes	Errores frecuentes
3. Ciclo de auto-juego y reentrenamiento	Refinar progresivamente la política del agente	Subdividido en pasos cíclicos		
3.1 Generar partidas automáticas	Que la DNN explore y produzca nuevas experiencias	Ejecutar partidas con el modelo actual; almacenar estados (X) y acciones (Y)	model.predict(), entorno AirBattle, listas o arrays para registrar episodios	Dataset demasiado repetitivo; falta de exploración de nuevas estrategias
3.2 Modificar dataset con reglas	Mejorar dataset aplicando heurísticas de desempeño	Revisar los estados y acciones; aplicar reglas programadas (ej: HEALTH<0.2 → CMD_BRAKE=1)	NumPy, Python, reglas if-else, reward shaping	Aplicar reglas incorrectas; sobre-corriger acciones; sesgo excesivo
3.3 Reentrenar modelo con dataset modificado	Ajustar la política de la DNN basándose en nuevas experiencias	Cargar modelo con load_model y entrenar con .fit() usando el dataset modificado	TensorFlow/Keras, .fit(), model.save/load	No normalizar entradas; entrenar con demasiadas pocas o demasiadas épocas; olvidar guardar modelo
3.4: Evaluación de métricas del modelo	Analizar qué tanto aprendió el modelo y cuáles entradas (inputs) tienen más impacto sobre las decisiones (outputs).	detectar variables inútiles, correlaciones falsas o comportamientos inesperados antes de lanzar otra ronda de autogeneración de partidas.	Análisis de impacto, Pesos Efectivos, Correlación Inputs outputs.	Confundir métricas de entrenamiento (loss, accuracy) con análisis de pesos, Interpretación equivocada de pesos, Omitir verificación con lógica del dominio.





Machine Learning



MANAGER ENTRENAMIENTO

El objetivo del script unificador es **orquestrar todas las etapas** para que el entrenamiento sea cíclico y autónomo:

- **Cargar el modelo inicial o crear uno nuevo.**
- **Generar partidas automáticas** (el modelo juega contra sí mismo).
- **Recolectar datasets** (ganador/perdedor, jugador 1/jugador 2).
- **Modificar y filtrar datasets** aplicando las reglas del dominio.
- **Reentrenar el modelo** con los datos actualizados.
- **Evaluar métricas** (impacto de entradas, correlaciones, pesos).
- **Guardar el modelo actualizado.**

Repetir el ciclo N veces o hasta que se cumplan criterios de mejora.

Funciones principales del MANAGER:

El script debería estructurarse en **módulos o funciones**, cada una llamando a los scripts ya existentes:

- `generar_partidas()` → llama al script que simula partidas y genera datasets.
- `modificar_datasets()` → procesa datasets y aplica reglas de consistencia.
- `entrenar_modelo()` → entrena o reentrena con el dataset combinado.
- `evaluar_metricas()` → ejecuta los scripts de análisis (impacto, correlación, pesos).
- `guardar_modelo()` → versiona el modelo (ej: `model_v2.h5`).

El **manager** las ejecuta en orden dentro de un bucle for ciclo in range(N):

Errores frecuentes en la unificación:

- **Rutas mal configuradas:** cada script puede estar guardando datasets en carpetas distintas; el manager debe unificar rutas.
- **Sobrescribir datasets o modelos:** si no se versionan correctamente, se pierden resultados previos.
- **No controlar la convergencia:** repetir ciclos sin analizar si realmente hay mejora puede llevar a sobreentrenamiento o estancamiento.
- **Falta de logs:** sin guardar un historial de métricas por ciclo, es imposible comparar la evolución.
- **Exceso de automatización:** no dejar puntos de control para revisión humana puede hacer que el modelo aprenda errores acumulados.

StockAI

X G Y G W: 1920 H: 1040

Start Training [1000.0 M]

G [X] BOTs Trainer

[1000.0 M]

G [X] BOTs Evaluator

[1000.0 M]

G [N] BOTs Genetic Lab

[1000.0 M]

CLAS TBL1

CLAS TBL2

CLAS TBL3

CLAS TBL4

Loop N° 54

GEN Loading (%) [0000000]

Total time (hrs) 55

Time/Loop (H) 59

Total DB Created 0

GEN DET Alive 0

CL DET Created 0

GENOT Trained 0

GENOT Selected [0000000]

GENOT Created 0

GENOT Alive [0000000]

GENOT MAX [0000000]

GENOT Threshold [0000000]

Pause

Exit

Training Room

Bot Room

DataTables

Pos.	Name	Gen	Flt Lev	Genes	Min	DF	DFE	DFB	DFP	Stall	Turnes	Break	FOX	Fox	Can	Can	Fia	Chaff	Mal	Fuel	Mal	Fia	Chaff	Coll
0	000 2023.11.30-20.36.03	010	10	53	72	65	45	17	0	1.0	1	1	1	0	22	9	1	1	0.04	1	0	0	0	0
1	001 2023.12.01-05.59.54	015	07	111	66	62	64	20	0	1.0	1	1	1	0	28	9	1	1	0.10	1	0	0	0	0
2	002 2023.11.30-21.23.23	012	02	177	59	56	52	28	0	0.0	0	0	1	0	21	8	0	0	0.05	1	0	0	0	0
3	003 2023.11.30-23.39.33	014	04	142	61	61	61	22	0	1.0	1	1	1	0	25	8	1	1	0.07	1	0	0	0	0
4	004 2023.12.01-05.47.02	015	02	154	61	62	52	25	0	0.0	0	0	1	1	20	7	1	1	0.10	1	0	0	0	0
5	005 2023.11.30-17.01.43	005	06	101	64	56	48	20	0	0.0	0	0	1	1	23	9	0	0	0.04	1	0	0	0	0
6	006 2023.11.30-23.44.41	014	06	162	65	63	56	23	0	0.0	0	0	1	1	29	9	1	1	0.11	1	0	0	0	0
7	007 2023.11.30-23.53.54	014	09	159	57	61	59	25	0	1.0	1	1	1	0	27	7	2	2	0.04	1	0	0	0	0
8	008 2023.12.01-05.53.37	015	05	156	58	65	63	23	0	0.0	0	0	1	1	29	10	1	0	0.08	1	0	0	0	0
9	009 2023.11.30-18.09.38	006	08	204	59	57	49	23	0	1.0	1	1	1	0	24	9	1	1	0.06	1	0	0	0	0
10	010 2023.11.30-20.33.53	010	09	143	60	59	49	20	0	0.0	0	0	1	1	23	7	0	0	0.10	1	0	0	0	0
11	011 2023.11.30-21.29.16	012	04	140	56	59	59	22	0	1.0	1	1	1	0	23	8	1	1	0.11	1	0	0	0	0
12	012 2023.12.01-02.49.02	017	07	132	60	68	60	19	0	1.0	1	1	1	0	28	11	2	1	0.04	1	0	0	0	0
13	013 2023.11.30-20.29.33	010	07	149	57	53	50	22	0	0.0	0	0	1	0	22	8	1	1	0.14	1	0	0	0	0
14	014 2023.11.30-21.20.27	012	01	152	56	55	54	24	0	0.0	0	0	1	0	24	6	1	1	0.09	1	0	0	0	0
15	015 2023.12.01-04.57.42	019	08	126	56	65	60	25	0	0.0	0	0	1	1	29	9	2	2	0.14	1	0	0	0	0
16	016 2025.02.12-15.35.12	046	56	67	64	15	1	2.0	0	5	1	1	1	27	11	0	0	0.05	0	0	0	0	0	0
17	017 2025.02.12-15.26.18	103	56	63	61	19	0	1.0	1	7	1	1	1	30	10	1	1	0.11	0	0	0	0	0	0
18	018 2023.02.13-05.51.33	90	60	69	65	17	0	0.0	2	6	1	1	1	32	11	2	2	0.04	1	0	0	0	0	0
19	019 0-075-F-10-2025.02.13-20.50.27	070	10	51	59	68	64	17	2	1.0	3	9	1	0	27	10	2	2	0.12	1	0	0	0	0
20	020 2025.02.13-09.18.33	76	58	65	65	16	0	0.0	2	6	1	1	1	27	11	2	2	0.09	0	0	0	0	0	0
21	021 0-077-F-09-2025.02.16-04.04.08	077	09	31	71	73	52	15	0	0.0	2	8	1	0	31	11	2	2	0.06	2	0	0	0	0
22	022 0-077-F-10-2025.02.16-04.01.29	077	10	30	67	75	60	12	0	0.0	2	7	1	0	29	12	1	1	0.23	1	0	0	0	0
23	023 2025.02.12-22.15.39	82	56	70	64	16	1	0.0	0	7	2	1	1	29	10	1	1	0.07	2	0	0	0	0	0
24	024 2023.11.30-17.10.03	005	05	176	56	54	45	21	0	2.0	2	1	1	25	9	0	0	0.07	1	0	0	0	0	0
25	025 0-080-F-09-2025.02.16-07.19.18	080	08	15	89	59	8	0.9	1	8	2	1.9	3	34	2	2	2	0.20	2	1	0	0	0	0
26	026 2023.11.30-20.47.46	014	07	154	55	65	65	19	0	0.0	0	0	1	1	31	7	1	2	0.05	1	0	0	0	0
27	027 0-075-F-10-2025.02.15-13.40.52	075	10	32	53	59	64	16	1	1.0	1	7	2	1	32	9	2	0	0.16	2	0	0	0	0
28	028 2023.11.30-19.31.58	009	10	190	54	67	48	20	0	0.0	0	0	1	1	25	8	0	0	0.10	1	0	0	0	0
29	029 0-078-F-09-2025.02.16-04.05.22	078	09	21	67	62	55	11	0	0.0	2	7	1	1	26	11	2	1	0.14	1	0	0	0	0
30	030 0-074-F-04-2025.02.14-06.37.38	074	04	37	54	61	53	20	1	0.0	1	7	1	0	25	9	1	1	0.11	1	0	0	0	0
31	031 0-078-F-07-2025.02.15-13.44.15	075	07	42	55	64	54	17	1	1.0	2	0	1	1	31	10	2	2	0.07	2	0	0	0	0
32	032 0-080-F-07-2025.02.16-07.15.47	080	07	9	67	73	49	15	0	0.0	2	8	1	1	16	11	2	2	0.17	1	0	0	0	0
33	033 0-044-F-09-2025.02.13-14.32.57	044	08	75	54	70	63	21	1	0.0	0	6	2	1	24	14	1	0	0.17	0	0	0	0	0
34	034 0-080-F-09-2025.02.16-07.22.42	080	09	10	60	45	42	30	1	0.0	1	9	1	0.2	24	7	2	2	0.20	1	0	0	0	0
35	035 0-081-F-10-2025.02.16-09.22.11	081	10	5	60	47	57	24	2	0.0	1	8	1	0	26	9	2	2	0.20	2	0	0	0	0
36	036 0-078-F-06-2025.02.16-04.55.17	078	06	27	63	59	55	17	0	0.0	1	6	0	0.1	27	10	1	1	0.11	1	0	0	0	0
37	037 0-081-F-06-2025.02.16-09.11.49	081	06	6	67	65	53	14	0	0.2	2	6	0	0	20	11	2	1	0.09	0	0	0	0	0
38	038 0-081-F-07-2025.02.16-09.14.24	081	07	6	50	59	50	12	0	0.0	1	8	1	0	28	10	1	1	0.08	2	0	0	0	0
39	039 0-078-F-09-2025.02.16-04.57.53	078	09	23	63	56	46	14	0	0.0	1	6	2	1.9	20	9	1	2	0.17	1	0	0	0	0
40	040 0-081-F-09-2025.02.16-09.17.00	081	08	9	56	45	55	24	0	0.0	1	7	0	0	31	9	1	1	0.09	2	0	0	0	0



Etapa 1: Crear Dataset Manual

Generar un dataset inicial que contenga ejemplos correctos de juego, es decir, pares **estado** → **acción** donde la acción tomada sea razonablemente buena. Este dataset servirá como base para entrenar la red neuronal antes de que aprenda a jugar por sí misma.

Tareas específicas:

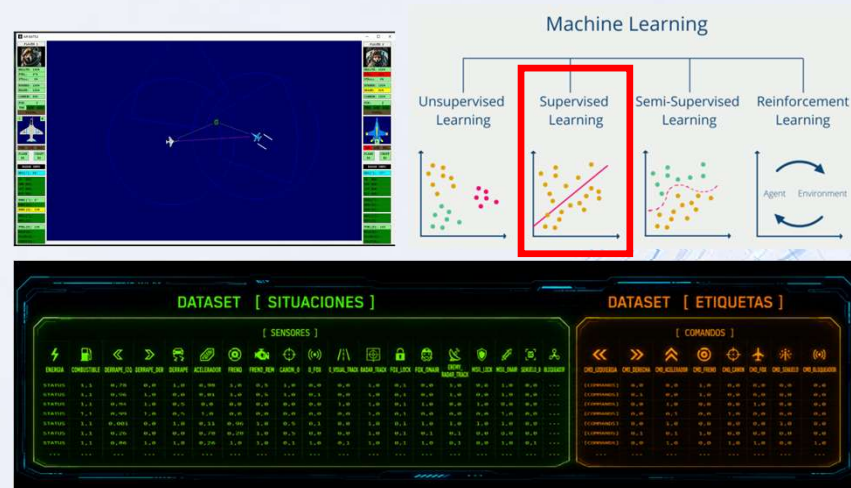
- Jugar partidas manualmente en AirBattle.
- Asegurarse de que los datos cubran distintos escenarios: atacar, esquivar, recolectar ítems, etc.
- Crear DataSet unificado.

Herramientas / Técnicas comunes:

- CSV o archivos Excel para almacenar datos.
- Pandas para manipular y limpiar los datasets.
- Scripts en Python para convertir logs del juego en tablas de estado y comandos.

Errores frecuentes:

- **Pocas partidas para generalizar:** El dataset solo refleja tus decisiones en un tipo de escenario, la red no aprende situaciones diversas.
- **Sesgo del jugador:** Si siempre juegas agresivo o pasivo, la DNN puede aprender una estrategia limitada.
- **Duplicación de datos:** Filas repetidas que sobre-representan ciertos estados y distorsionan el aprendizaje.
- **Falta de exploración de casos extremos:** No incluir situaciones críticas (salud baja, enemigos cercanos, pocos recursos) que son importantes para que la DNN aprenda a reaccionar.
- **Falta de diversidad de escenarios:** Si todas las partidas ganadas se obtienen con la misma estrategia, la DNN no aprenderá a adaptarse a situaciones distintas.



```
SCRIPT >>> "TRAIN02_UNIFICADOR_DATASET.py"
```



Etapas 2: Entrenar modelo inicial

Que la DNN aprenda a **imitar las decisiones correctas de las partidas manuales**, utilizando los datasets generados por AirBattle (ganador y perdedor). Esta etapa establece la **política inicial** del agente antes de que comience a jugar contra sí mismo y a refinarse.

Tareas específicas:

Definir la arquitectura de la DNN:

Entradas: 40 sensores del avión.

Capas ocultas: p.ej., 2 capas densas con 10 neuronas cada una.

Salidas: 8 comandos (CMD_LEFT, CMD_RIGHT, ...).

Preparar los datos:

Leer los CSV de ganador y perdedor.

Normalizar las entradas (HEALTH, FUEL, etc.) para que los valores estén en rangos compatibles con la función de activación (sigmoid).

Opcional: ponderar los ejemplos de ganadores más alto que los perdedores para reforzar buenas acciones.

Compilar el modelo:

Seleccionar **optimizador** (adam, sgd).

Seleccionar **función de pérdida** adecuada (binary_crossentropy) porque las salidas son acciones binarias.

Entrenar con .fit():

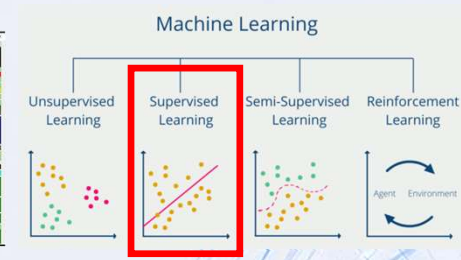
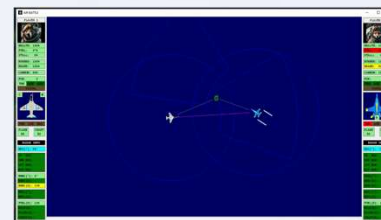
Ajustar el modelo usando el dataset manual.

Definir número de épocas, tamaño de batch y validación si es necesario.

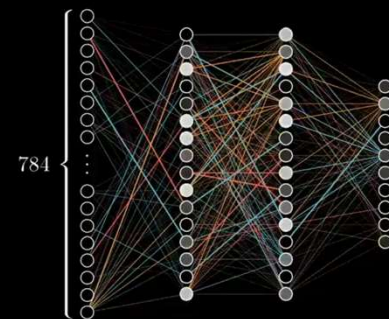
Guardar el modelo entrenado:

Para poder cargarlo en etapas posteriores y continuar el entrenamiento iterativo.

`model.save("airbattle_model_initial.keras")`



Training in progress. . .



SCRIPT >>> "TRAIN03_AIRBATTLE_ENTRENAMIENTO.py"



Etapa 2: Entrenar modelo inicial

Que la DNN aprenda a **imitar las decisiones correctas de las partidas manuales**, utilizando los datasets generados por AirBattle (ganador y perdedor). Esta etapa establece la **política inicial** del agente antes de que comience a jugar contra sí mismo y a refinarse.

Errores frecuentes

Overfitting por pocas partidas:

La DNN memoriza patrones específicos de las partidas manuales y no generaliza bien a nuevas situaciones.

Dataset desequilibrado:

Si hay muchos más ejemplos de perdedor que de ganador, la DNN puede aprender estrategias incorrectas.

No normalizar las entradas:

Valores como HEALTH o FUEL en escalas muy diferentes pueden hacer que la DNN aprenda lentamente o mal.

Arquitectura inadecuada:

Muy pocas neuronas/capas → incapaz de aprender patrones complejos.

Muy grande → entrenamiento lento y riesgo de overfitting.

Funciones de activación incorrectas:

Usar relu en la capa de salida para acciones binarias puede producir resultados no binarios; se recomienda sigmoid o softmax según el enfoque.

Batch o número de épocas inapropiado:

Demasiadas épocas → sobreajuste; pocas épocas → aprendizaje incompleto.

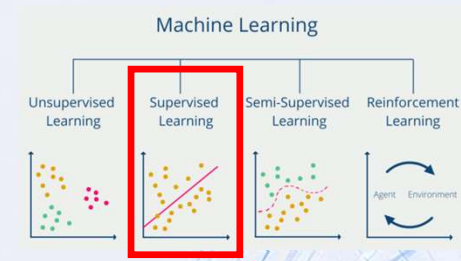
Tamaño de batch demasiado grande → poca variabilidad por actualización; demasiado pequeño → entrenamiento ruidoso.

No validar durante entrenamiento:

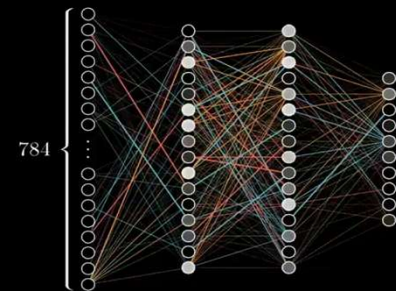
Sin un dataset de validación, no se detecta si el modelo está aprendiendo correctamente o simplemente memorizando ejemplos.

Ignorar secuencia temporal:

.fit() trata cada fila como independiente; si el juego tiene secuencia de estados, la DNN podría no capturar ciertas dinámicas temporales (esto se corrige más adelante con reglas o RL).



Training in progress...



```
SCRIPT >>> "TRAIN03_AIRBATTLE_ENTRENAMIENTO.py"
```




Etapas 3.1: Generar partidas automáticas

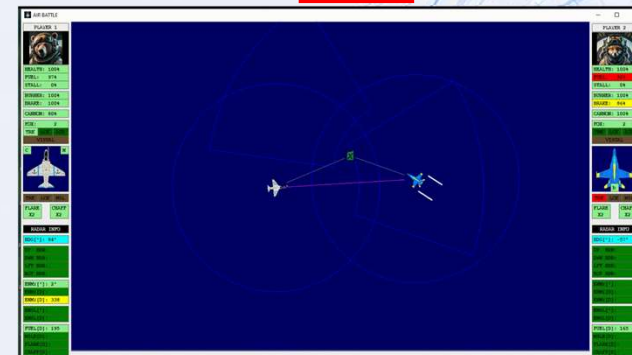
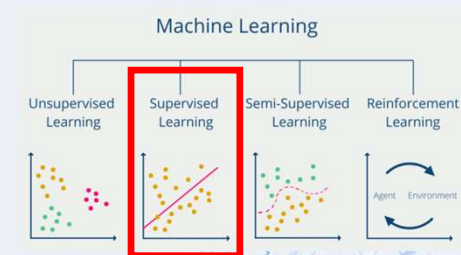
Permitir que la DNN juegue contra sí misma para **producir nuevas experiencias** que luego se usarán para mejorar la política del agente. Esto permite que la red aprenda a enfrentarse a situaciones más diversas que las observadas en las partidas manuales.

Tareas específicas:

- **Cargar el modelo entrenado inicialmente:** Se usa el modelo entrenado en la Etapa 2.
- **Configurar el entorno AirBattle:**
 - Inicializar la partida para que ambos jugadores sean controlados por la DNN.
 - Registrar cada frame los estados (X) y las acciones tomadas (Y).
- **Ejecutar la partida automáticamente:**
 - En cada paso del juego, predecir las acciones usando `model.predict(state)`.
 - Aplicar un umbral (p.ej., >0.5) para decidir si activar cada comando.
 - Guardar estados y comandos en arrays o listas.
- **Al finalizar la partida:** AirBattle genera dos datasets automáticamente: uno para el **ganador** y otro para el **perdedor**.

Errores frecuentes

- **Dataset demasiado repetitivo:** Si la DNN juega siempre igual, los estados y acciones generados no aportan información nueva.
- **Falta de exploración:** La DNN puede quedarse atrapada en estrategias subóptimas ("política local"), porque solo replica lo aprendido.
- **Desalineación de estados y acciones:** Guardar un estado con la acción de un frame diferente puede corromper el dataset.
- **Errores de sincronización con el juego:** Latencia o retrasos en la predicción pueden generar acciones aplicadas tarde o inconsistentes.
- **Sobrepeso de un jugador:** Si uno de los jugadores DNN es consistentemente más fuerte, el dataset del perdedor podría ser demasiado amplio y no reflejar situaciones útiles.
- **Fallos en la predicción de la DNN:** Salidas continuas sin binarizar correctamente los comandos pueden generar datos incorrectos.
- **Archivos corruptos o no guardados:** Partida interrumpida por error del juego o fallo de disco puede provocar datasets incompletos.
- **Ignorar diversidad de escenarios:** Si siempre se inicializa la partida de la misma manera, los datos generados no cubren todas las posibles situaciones del juego.



```
SCRIPT >>> "TRAIN01_EJECUTADOR_AIR_BATTLE.py"
```

```
SCRIPT >>> "COMMS01_KB_PLAYER.py"
```

```
SCRIPT >>> "COMMS02_AI_PLAYER.py"
```



Etapa 3.2: OPTIMIZACION del DATASET

Anti-Leakage Sanitization

Objetivo: Detectar y neutralizar sensores que funcionan como un "espejo" del comando que se quiere predecir, para que la red aprenda la relación real estado → acción y no un atajo trivial.

Concepto clave: Causal Confusion

- Es un proceso de higiene de datos que se aplica *después* de que las políticas ya resolvieron los comandos.
- En la literatura de Imitation Learning se lo conoce como mitigar la **Causal Confusion** (De Haan et al., 2019): cuando un sensor está prácticamente correlacionado 1 a 1 con el comando que se quiere predecir, la red aprende a copiarlo en vez de aprender la causa real de la acción.
- Esa "muleta" desaparece —o directamente falla— apenas el modelo tiene que volar solo, sin ese comando previo disponible como atajo.
- Es una de las causas más comunes de que un modelo funcione perfecto en entrenamiento y colapse en inferencia.

Tipos de fuga a buscar

- **Sensores de retorno directo:** variables que en la práctica replican el comando del paso anterior. Aportan cero información causal nueva y deberían neutralizarse sistemáticamente.
- **Sensores acoplados por construcción física:** variables que crecen o decrecen *mientras* se mantiene presionado cierto comando, generando una correlación espuria fuerte aunque no sean un espejo exacto. Aquí la solución no siempre es "apagar siempre": conviene preguntarse si, por debajo de cierto umbral de relevancia, el sensor es solo ruido correlacionado (se neutraliza), pero por encima de ese umbral pasa a ser una señal de seguridad real (se preserva, e incluso puede justificar forzar un override de seguridad sobre el comando).

Tareas específicas

- Auditar el esquema de sensores en busca de candidatos a fuga (buscar correlaciones altas entre sensor y comando correspondiente).
- Clasificar cada candidato: ¿es una fuga pura (se neutraliza siempre) o una fuga condicional (se neutraliza solo por debajo de un umbral de relevancia física)?
- Registrar estadísticas de cuántas veces se aplicó cada neutralización/override, para poder auditarlas.

Errores frecuentes en esta etapa

- **No distinguir fuga pura de fuga condicional:** apagar un sensor que en niveles extremos es información de seguridad real elimina señal valiosa, no solo ruido.
- **Aplicar la sanitización antes de que las políticas terminen de resolver los comandos:** el orden importa, porque la sanitización necesita el comando ya definitivo para decidir si corresponde un override.
- **Ignorar esta etapa por completo:** es la causa número uno de que un modelo entrenado con altísima métrica offline se comporte de forma errática al volar de forma autónoma.

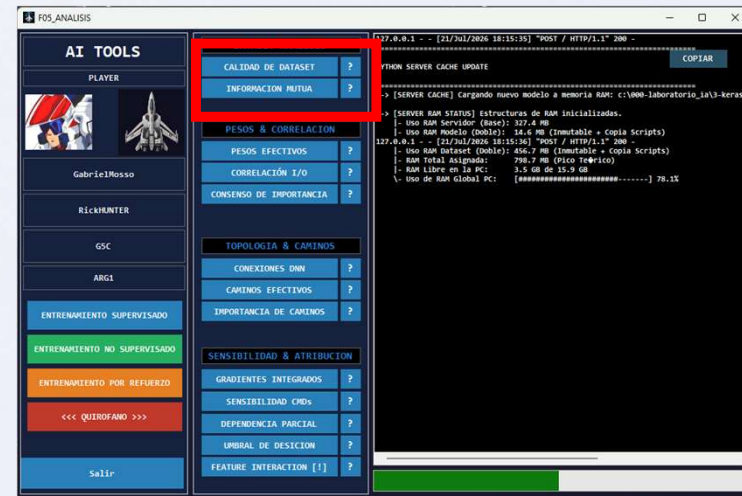
```
SCRIPT >>> "TRAIN02_UNIFICADOR_DATASET.py"
```



Etapas 3.3: Análisis del DATASET.

Script: CHECK_DA_01_CALIDAD_DEL_DATASET.py

Objetivo general: auditar la calidad del dataset de entrenamiento desde múltiples ángulos: balance de clases, ruido en las decisiones, estabilidad temporal y complejidad de las etiquetas.



SCRIPT >>> "CHECK_DA_01_CALIDAD_DEL_DATASET.py"



Etap 3.3: Análisis del DATASET.

Figura 1 — Balance de Comandos + Co-ocurrencia

Subplot izquierdo: Balance General de Comandos (barh)

Muestra la **frecuencia de uso (%)** de cada comando (CMD_*) sobre el total de filas del dataset.

Cálculo: $\text{df}[\text{columnas_comando}].\text{mean}() * 100$.

Para qué se usa: detectar **desbalance de clases**.

Un comando usado <1% de las veces = casi sin señal positiva para entrenar.

Riesgo directo: recall bajo en esa clase específica.

Subplot derecho: Heatmap de Co-ocurrencia

Matriz de **probabilidad condicional**: $P(\text{columna}=1 \mid \text{fila}=1)$.

Cálculo: $(\text{activaciones simultáneas}) / (\text{activaciones de la fila})$.

No es simétrica — se lee direccionalmente: "cuando aprieto FILA, ¿qué % de las veces también aprieto COLUMNA?".

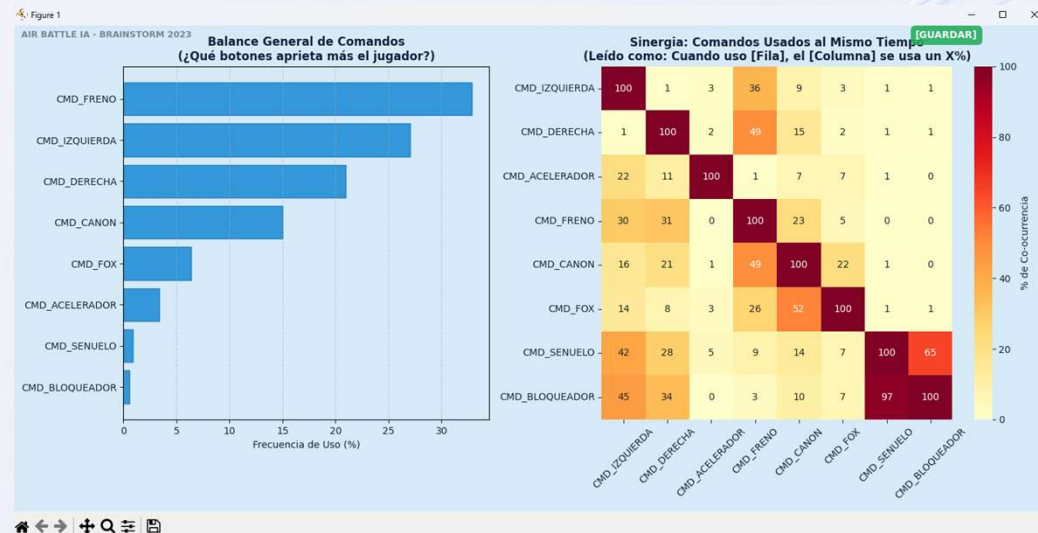
Para qué se usa: detectar **sinergias reales de gameplay** (ej. FRENO + DERECHA en curvas).

Importa porque en multilabel el modelo debe aprender combinaciones, no solo clases aisladas.

Relación con otras figuras:

Es la base de la **Figura 5** (detalla combinaciones exactas).

Se resume numéricamente en la **Figura 4** (cardinalidad, sin decir cuáles se combinan).





Etapas 3.3: Análisis del DATASET.

Figura 2 — Target Conflict (Ruido en las Decisiones)

Usa **KDTree (KNN)** sobre sensores normalizados (z-score) para encontrar el vecino más cercano de cada fila en el espacio de estado.

Si distancia $< 0.5 \rightarrow$ se considera "misma situación".

Compara si el jugador tomó **la misma decisión** en ambos casos.

Subplot izquierdo: Torta (Consistencia del Jugador)

% de pares "casi idénticos" con decisiones **consistentes** vs **contradictorias**.

Subplot derecho: Panel explicativo

Define el criterio: **>15% de conflicto = ruido significativo**.

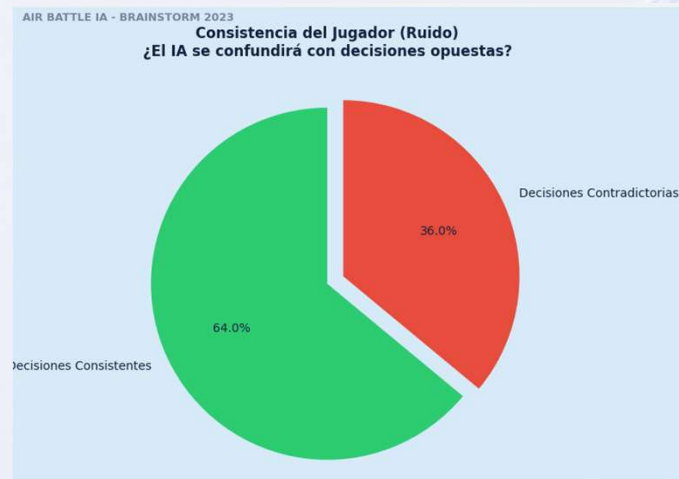
Para qué se usa: medir si el dataset enseña **dos respuestas correctas para la misma pregunta**.

Esto pone un **techo duro** a la accuracy alcanzable, sin importar qué tan bueno sea el modelo.

Relación con otras figuras:

Distinto de la Figura 1: no mide balance, mide **contradicción etiqueta-vs-estado**.

⚠ **Nota técnica:** el umbral 0.5 es fijo y arbitrario respecto a 40 sensores normalizados — conviene calibrarlo si el % de conflicto sale sospechosamente alto o bajo.





Eta3a 3.3: Análisis del DATASET.

Figura 3 — Estabilidad Temporal (Data Drift)

(Proceso 5)

Divide el dataset (ya en orden cronológico) en **50 bloques secuenciales**.

Grafica frecuencia (%) de cada CMD_* y SIT_* bloque a bloque.

Líneas **sólidas** = comandos | Líneas **punteadas** = situaciones.

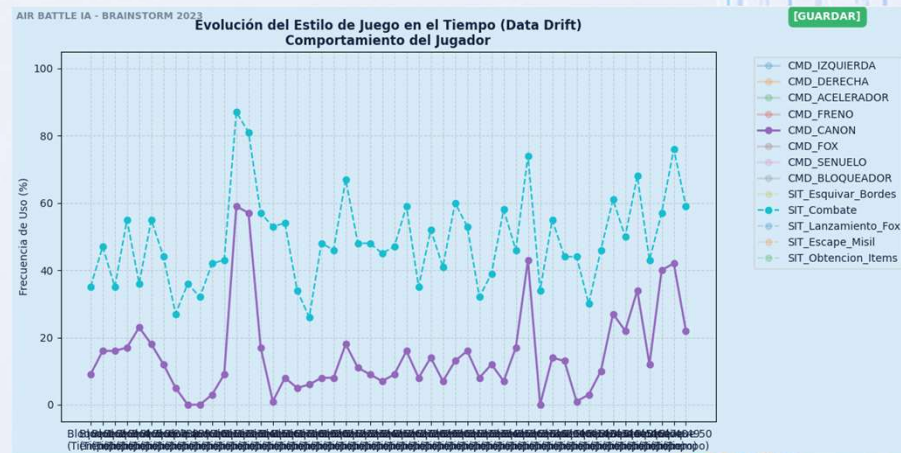
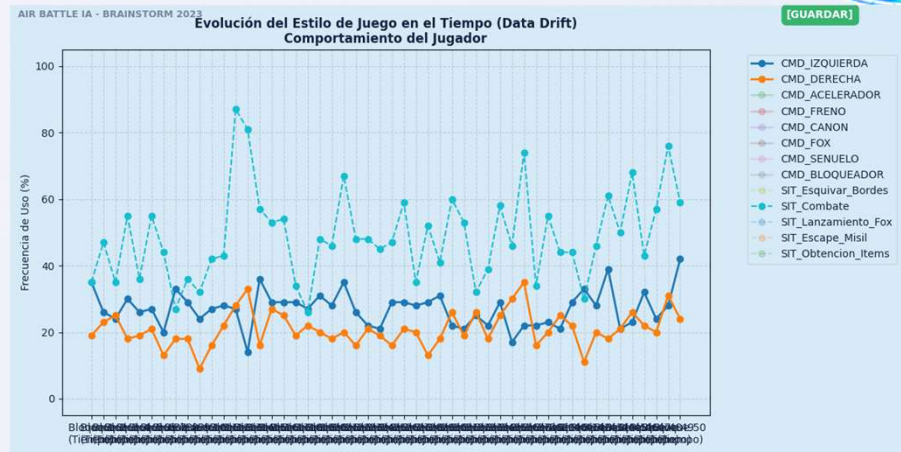
Interactivo: click en la leyenda oculta/muestra series (por defecto solo visibles: CMD_IZQUIERDA, CMD_DERECHA y las SIT_*).

Para qué se usa: detectar si el **estilo de juego cambió con el tiempo**.

Ej: jugador mejoró, cambió de estrategia, o se mezclaron sesiones distintas.

Relación con otras figuras:

Complementa temporalmente a la **Figura 1**: la Fig. 1 da el promedio global, esta muestra si ese promedio es estable o un artefacto de mezclar comportamientos distintos.





Etapla 3.3: Análisis del DATASET.

Figura 4 — Complejidad de Decisiones (Cardinalidad)

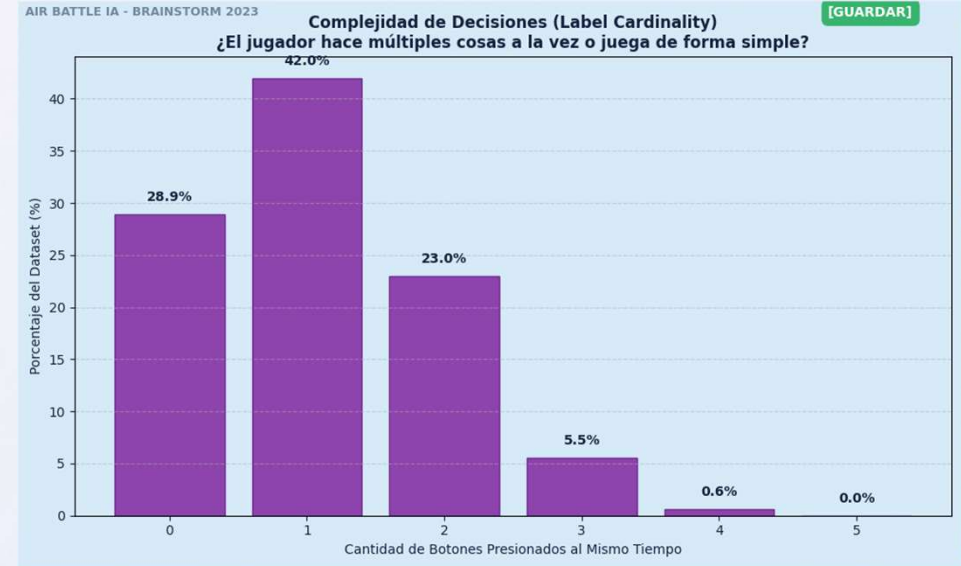
Histograma de `df[columnas_comando].sum(axis=1)`: cuántos botones se presionan **simultáneamente** por fila (0, 1, 2, 3...).

Métrica estándar de "label cardinality" en literatura multilabel.

Para qué se usa: dimensionar qué tan compleja es la salida que el modelo debe predecir por muestra.

Relación con otras figuras:

Resume en un solo número lo que la **Figura 1** (heatmap) muestra en detalle par a par. Precede a la **Figura 5**, que detalla combinaciones exactas.





Etap 3.3: Análisis del DATASET.

Figura 5 — Combinaciones Activas (Powerset Imbalance)

Concatena comandos activos por fila en un string (ej. "IZQUIERDA + FOX").

Excluye filas sin ningún comando activo ("NINGUNO").

Calcula frecuencia de cada combinación única sobre el subconjunto de filas activas.

Grafica:

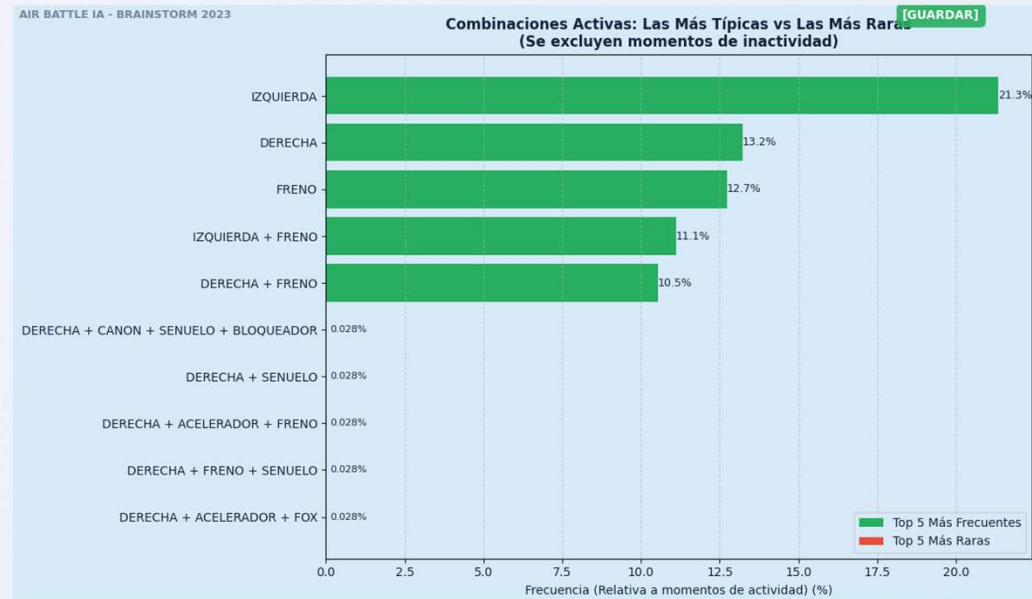
- Top 5 más frecuentes
- Top 5 más raras

Para qué se usa: detectar desbalance del "powerset".

Aunque cada etiqueta individual esté balanceada, las **combinaciones específicas** pueden ser extremadamente raras. Esas combinaciones raras son las que el modelo casi nunca aprenderá a predecir bien.

Relación con otras figuras:

Mayor nivel de detalle que la **Figura 4** (que solo cuenta *cuántos*, no *cuáles*).





Etapa 3.3: Análisis del DATASET.

Figura 6 — Frecuencia de Situaciones de Juego

Barra con la frecuencia (%) de las 5 columnas SIT_*, derivadas heurísticamente de sensores (definidas en el código, líneas 157-161):

SIT_Esquivar_Bordes

- Condición: distancia a cualquiera de los 4 bordes del mapa (SUP/INF/IZQ/DER) < 1.
- Implica: el jugador está **al límite del área jugable** y necesita maniobrar para no salirse o chocar contra el borde.

SIT_Combate

- Condición: VISUAL_TRACK == 1 o RADAR_TRACK == 1.
- Implica: el jugador tiene a un **enemigo detectado**, ya sea visualmente o por radar. Es la condición "hay un enemigo cerca", la más general de las situaciones ofensivas.

SIT_Lanzamiento_Fox

- Condición: RADAR_TRACK == 1 o FOX_LOCK == 1 o FOX_ONAIR == 1.
- Implica: el jugador está en proceso de **preparar o ejecutar un disparo tipo "Fox"** (misil guiado) — desde el tracking por radar hasta el lock-on o el misil ya en vuelo.

SIT_Escape_Misil

- Condición: ENEMY_RADAR_TRACK == 1 o MSL_LOCK == 1 o MSL_ONAIR == 1.
- Implica: el jugador está siendo **detectado o atacado** por el enemigo (radar enemigo activo, lock enemigo sobre el jugador, o misil enemigo en vuelo).

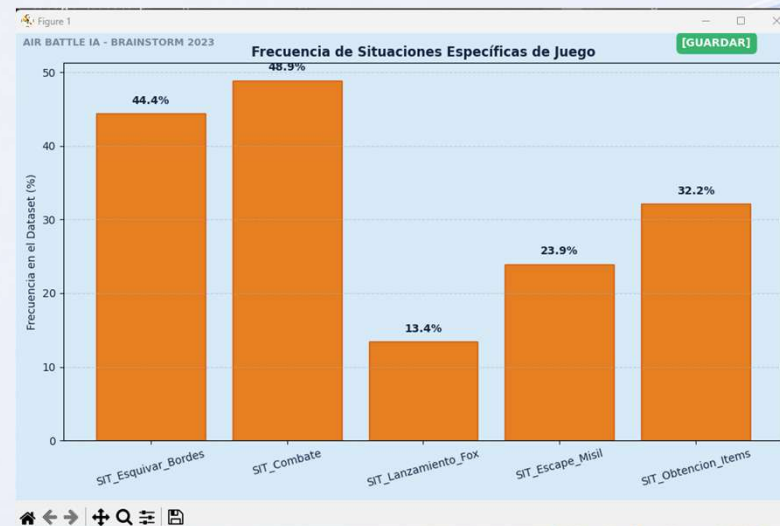
SIT_Obtencion_Items

- Condición: distancia a cualquier ítem (Combustible, Misil, Señuelo, Bloqueador) < 1.
- Implica: el jugador está **cerca de recoger un power-up/recurso** en el mapa (recarga de combustible, munición, contramedidas, etc.).

Nota: las condiciones no son mutuamente excluyentes — un mismo frame puede activar varias SIT_* a la vez (ej. estar en combate Y cerca de un borde). Por eso las frecuencias del gráfico no suman 100%.

Para qué se usa: dar **contexto situacional** al análisis.

- Las Figuras 1-5 hablan de qué *comandos* aprieta el jugador.
- Esta figura habla de en qué *situaciones* se encuentra.
- Clave para detectar situaciones raras pero críticas (ej. baja frecuencia de SIT_Escape_Misil = pocos ejemplos para entrenar esa conducta).





Etapas 3.3: Análisis del DATASET.

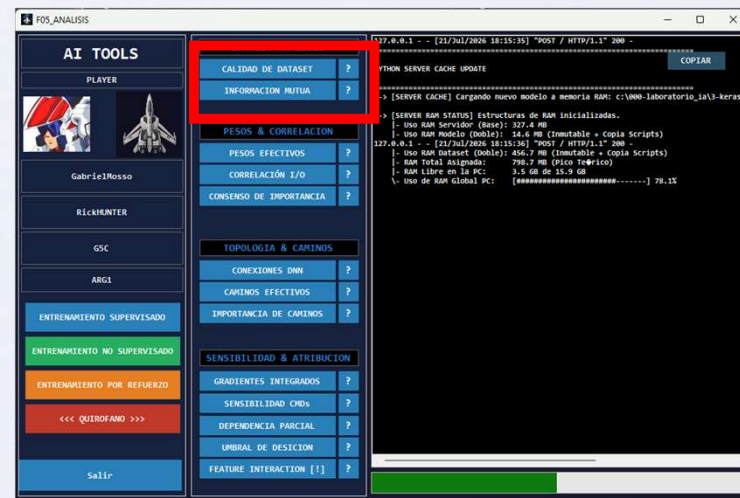
Script: CHECK_DA_01_CALIDAD_DEL_DATASET.py

Objetivo general: auditar la calidad del dataset de entrenamiento desde múltiples ángulos: balance de clases, ruido en las decisiones, estabilidad temporal y complejidad de las etiquetas.

Pregunta que responde	Figura(s)
¿Está balanceado cada comando individualmente?	1 (izq)
¿Qué comandos se usan juntos?	1 (der), 5
¿Hay etiquetas contradictorias para el mismo estado?	2
¿Cambié el estilo de juego en el tiempo?	3
¿Cuántos botones a la vez, en promedio?	4
¿Qué combinaciones exactas son raras?	5
¿En qué contexto/situación ocurre todo esto?	6

Conclusión: en conjunto arman un diagnóstico completo de calidad para dataset multilabel:

- **Balance de clases** (Fig. 1)
- **Ruido / etiquetas ambiguas** (Fig. 2)
- **Estacionariedad** (Fig. 3)
- **Complejidad de la salida** (Fig. 4-5)
- **Cobertura de escenarios** (Fig. 6)



SCRIPT >>> "CHECK_DA_01_CALIDAD_DEL_DATASET.py"

SCRIPT >>> "CHECK_DA_02_SENSOR_MUTUAL_INFORMATION.py"



Etapas 3.3: Análisis del DATASET.

Análisis de Redundancia entre Sensores (Mutual Information) — Descripción Técnica de Gráficas

Script: CHECK_DA_02_SENSOR_MUTUAL_INFORMATION.py

Objetivo Principal del Script:

Medir la **redundancia de información entre pares de sensores** (relación *input-input*), a diferencia de un análisis de "importancia de entradas" que mide relevancia *input-output* (cuánto influye cada sensor en la salida de la red).

Es la mitad complementaria de un flujo de **feature selection**:

- Un sensor puede ser muy **relevante** para la salida, pero si es muy **redundante** con otro sensor, uno de los dos podría eliminarse sin perder poder predictivo → red más simple y rápida.
- Un sensor poco relevante pero también poco redundante (independiente) puede seguir aportando información marginal útil.

Combina **tres medidas de dependencia estadística**, cada una sensible a un tipo distinto de relación entre dos sensores:

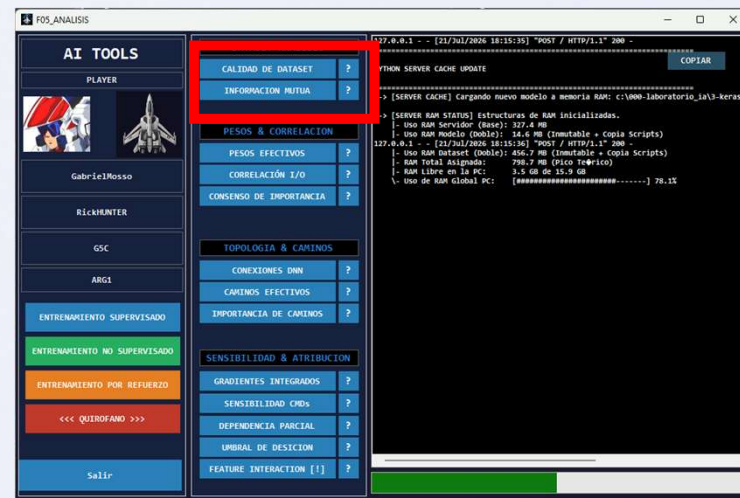
Pearson → relaciones **lineales** (con signo).

Spearman → relaciones **monótonas** (con signo), robusta a outliers y no linealidades monótonas.

Información Mutua (MI) → relaciones **generales**, incluye no lineales y no monótonas (estimador KSG de sklearn).

Calcula además un **umbral de significancia** para la MI mediante **prueba de permutación** (baraja cada sensor de forma independiente para destruir la dependencia real y mide la MI "de ruido" resultante), evitando reportar cualquier valor de MI como automáticamente relevante.

La diferencia entre MI y el máximo de |Pearson|/|Spearman| ("**brecha de no-linealidad**") permite detectar pares redundantes que un análisis de correlación clásico **no detectaría**.



SCRIPT >>> "CHECK_DA_02_SENSOR_MUTUAL_INFORMATION.py"

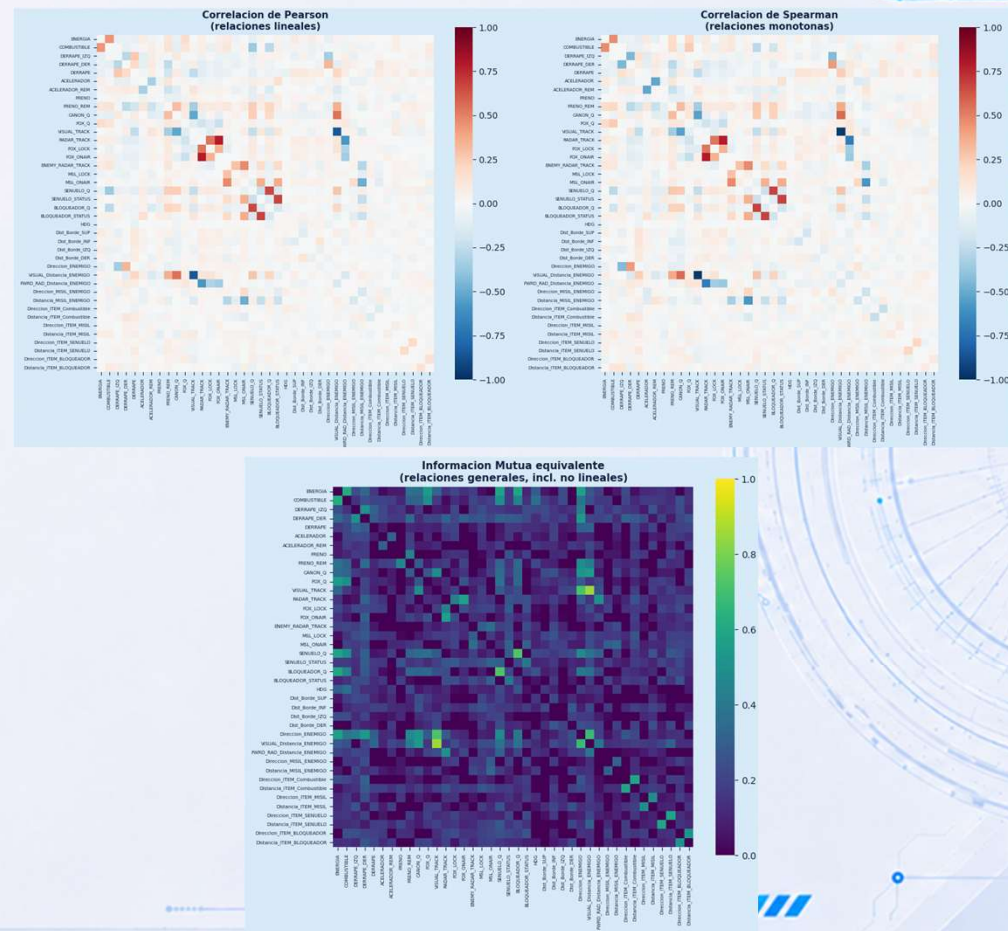
Figura 1 — Comparativa de Medidas de Dependencia

- Pearson** (RdBu_r, rango -1 a 1) → relaciones lineales.
- Spearman** (RdBu_r, rango -1 a 1) → relaciones monótonas.
- MI-equivalente** (viridis, rango 0 a 1) → relaciones generales (incluye no lineales).

Para qué se usa: dar una vista panorámica y comparar, sensor por sensor, si una relación es capturada por las 3 medidas o solo por la MI (lo cual ya insinúa no-linealidad).

Relación con otras figuras:

- Es la base visual de la **Figura 2** (que calcula la diferencia explícita entre estos paneles).
- Los valores de MI-equivalente de este heatmap alimentan directamente las **Figuras 4 y 5**.





Etapas 3.3: Análisis del DATASET.

Figura 2 — Detección de Redundancia No-Lineal

Panel izquierdo: Heatmap de "brecha no-lineal"

Calcula MI-equivalente - $\max(|\text{Pearson}|, |\text{Spearman}|)$ para cada par de sensores.

Cuanto más alto el valor, más redundancia existe entre ese par que **Pearson y Spearman no logran capturar**.

Panel derecho: Top 5 pares con mayor brecha (barh)

Lista los 5 pares de sensores con mayor redundancia "invisible" para las correlaciones clásicas.

Para qué se usa: identificar pares de sensores que **parecen independientes bajo un análisis de correlación tradicional**, pero que en realidad comparten información de forma no lineal — el tipo de redundancia que un ingeniero podría pasar por alto si solo mirara Pearson.

Relación con otras figuras:

Es un "zoom" sobre la diferencia entre los paneles 1 y 3 de la **Figura 1**.

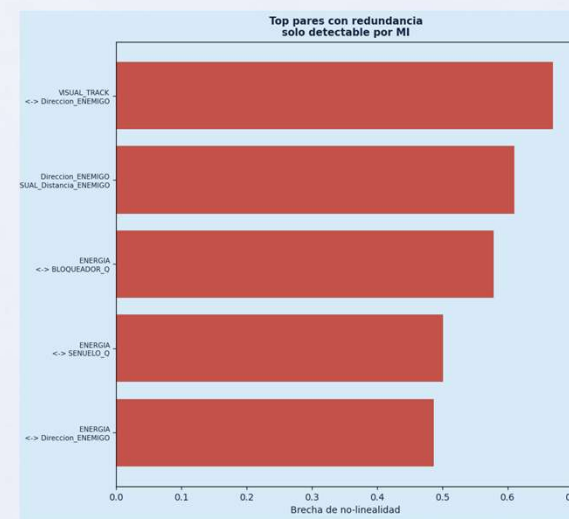
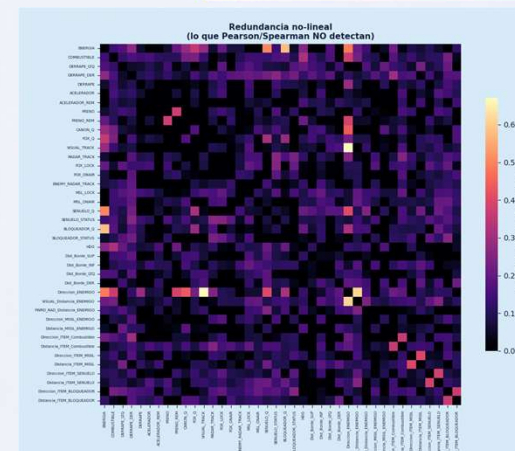


Figura 4 — Red de Redundancia entre Sensores

Grafo circular interactivo: cada sensor es un nodo distribuido en círculo.

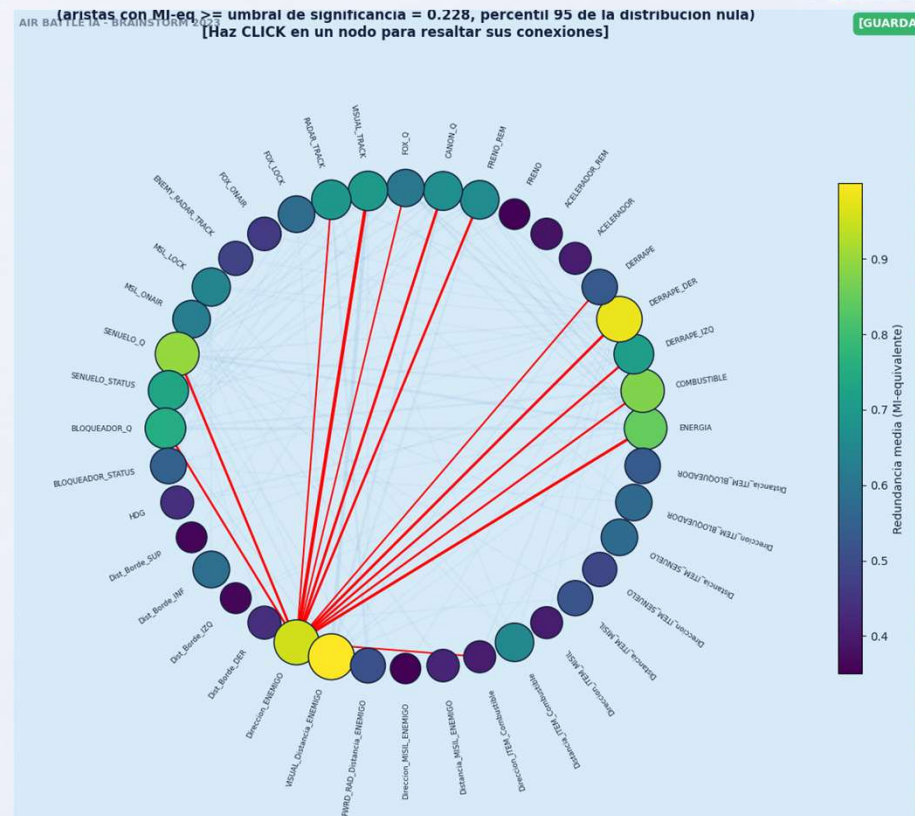
Tamaño y color de nodo: proporcional a la redundancia media del sensor con el resto ($MI_{corr.mean}(axis=1)$, normalizada 0-1).

Interactivo: click en un nodo resalta en rojo sus conexiones y atenúa el resto.

Para qué se usa: dar una vista de **estructura global** de la redundancia — qué sensores actúan como "hubs" muy conectados (candidatos fuertes a poda) vs. cuáles quedan aislados (aportan información única).

Relación con otras figuras:

Usa el mismo umbral de significancia que colorea la **Figura 5** (izquierda).
Es la versión "de red" de lo que la **Figura 5** (derecha) resume como ranking simple.





Etap 3.3: Análisis del DATASET.

Figura 5 — Rankings de Redundancia e Independencia

Panel izquierdo: Top 10 pares más redundantes (barh)

Ordenado por MI-equivalente descendente.

Color **verde** = estadísticamente significativo (supera el umbral de permutación) | **gris** = no significativo.

Línea vertical roja punteada = umbral de significancia.

Panel derecho: Sensores más independientes (barh)

Top 12 sensores con **menor** MI-equivalente promedio respecto al resto (mean_mi).

Para qué se usa: dar una lista accionable y priorizada:

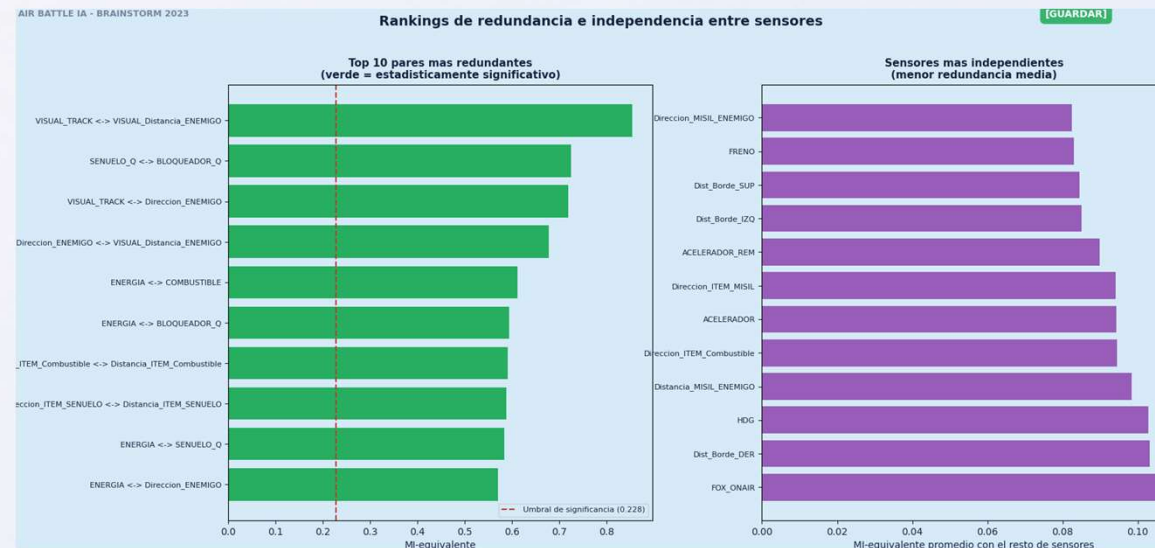
Los pares del panel izquierdo son **candidatos a eliminar uno de los dos** (redundancia alta y confirmada estadísticamente).

Los sensores del panel derecho son los que **más vale la pena conservar**, porque aportan información que ningún otro sensor replica.

Relación con otras figuras:

Resume en formato de ranking simple lo que la **Figura 4** muestra como red completa.

El "significativo/no significativo" del panel izquierdo usa el mismo umbral calculado en el Proceso 5 que define las aristas de la **Figura 4**.





Etapas 3.3: Análisis del DATASET.

Análisis de Redundancia entre Sensores (Mutual Information) — Descripción Técnica de Gráficas

Script: CHECK_DA_02_SENSOR_MUTUAL_INFORMATION.py

Objetivo Principal del Script:

Medir la **redundancia de información entre pares de sensores** (relación *input-input*), a diferencia de un análisis de "importancia de entradas" que mide relevancia *input-output* (cuánto influye cada sensor en la salida de la red).

Pregunta que responde

¿Cómo se comparan Pearson, Spearman y MI sensor por sensor?

¿Qué redundancia es invisible para correlación clásica?

¿Cuál es la estructura global de redundancia entre todos los sensores?

¿Qué pares específicos conviene podar y qué sensores conviene conservar?

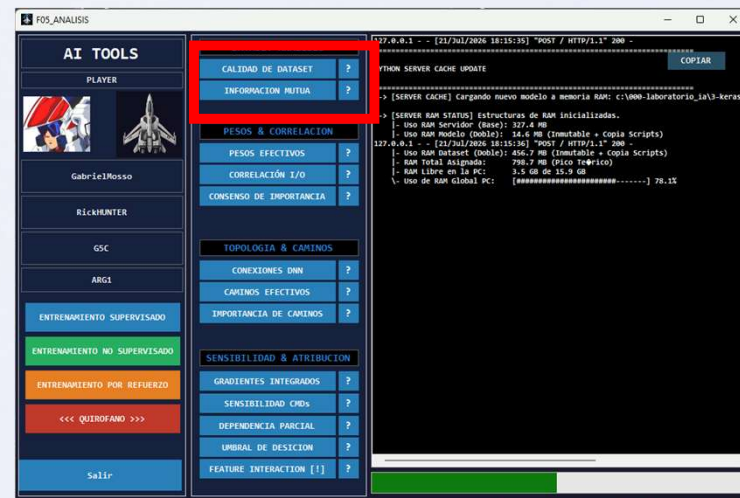
Figura(s)

1

2

4

5



SCRIPT >>> "CHECK_DA_02_SENSOR_MUTUAL_INFORMATION.py"

Conclusión: en conjunto arman un pipeline de **feature selection** basado en redundancia: comparación de métricas (1), detección de no-linealidad oculta (2), estructura relacional global (4) y recomendaciones accionables de poda/conservación (5), todo respaldado por un umbral de significancia estadística en vez de valores de MI crudos sin contexto.

Objetivo:

Tareas específicas:

Unificar datasets de partidas ganadoras:

- Recorrer la carpeta de archivos .Battle generados por AirBattle.
- Concatenar todos los archivos en un único dataset unificado (Dataset_Unificado.txt).

Depuración de datos:

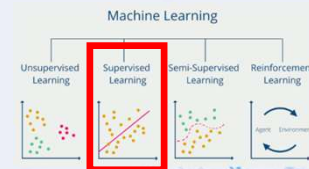
- Leer el dataset unificado y **balancear** las diferentes situaciones para que cada una tenga una **proporción** representativa.
- Contar estadísticas de filas totales, descartadas y conservadas.

Aplicar reglas heurísticas sobre los datos:

- **Ejemplo 1:** Si $STALL > 0.7$, desactivar comandos de giro (`CMD_LEFT` y `CMD_RIGHT`) para evitar perder control del avión.
- **Ejemplo 2:** (ya incluido) filas con todos los comandos en cero se descartan automáticamente. Estas reglas **mejoran la calidad del dataset**, asegurando que la DNN no aprenda comportamientos peligrosos o inútiles.

Errores frecuentes en esta etapa

- **Reglas demasiado estrictas:** Aplicar correcciones excesivas puede eliminar variabilidad útil y limitar la capacidad de la DNN de aprender estrategias alternativas.
- **Descartar filas útiles:** Filas donde todos los comandos son cero podrían representar situaciones de espera estratégica; descartarlas siempre puede eliminar casos válidos.

[illegible]

```
SCRIPT >>> "TRAIN02_UNIFICADOR_DATASET.py"
```

Un dataset de partidas crudas nunca está listo para entrenar una DNN.
Contiene:

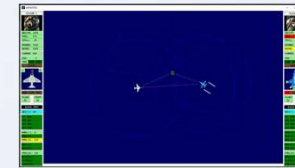
- Ruido humano
- Filas sin información
- Atajos espurios
- Desequilibrios entre situaciones frecuentes y raras.

Antes de entrar en las técnicas, una distinción conceptual que atraviesa toda la etapa:

- **Regla:** Una condición puntual y aislada (si X entonces Y).
- **Política:** Un conjunto organizado de reglas que persigue un objetivo coherente (seguridad, navegación, combate, recuperación...).
- **SIT (Situación):** ¿La condición física ocurrió? (ej.: hay una amenaza dentro de rango)
- **COR (Corrección):** Dado que ocurrió, ¿hizo falta además modificar el comando para resolverla?

Con esto en mente, la Etapa 3 se organiza en las siguientes presentaciones:

- Ingestion y Dataset Cleaning
- Policy Engineering (inyección de conocimiento experto)
- Anti-Leakage Sanitization
- Dataset Filtering y Dataset Balancing
- Label Engineering
- Action Masking
- Dataset Randomization y Dataset Validation

[illegible]

```
SCRIPT >>> "TRAIN02_UNIFICADOR_DATASET.py"
```


The screenshot displays the 'Situaciones' (Scenarios) screen in the game. It is divided into two main sections: 'DATASET [SITUACIONES]' on the left and 'DATASET [ETIQUETAS]' on the right. The left section features a 'SELECCIONAR' button and a table with columns for various sensors (FUEGA, GASES, etc.) and rows for different scenarios (SITUACION_1 to SITUACION_10). The right section features a 'COMANDOS' button and a table with columns for different commands (COMANDO_1 to COMANDO_10) and rows for different scenarios. The interface is dark-themed with green and yellow highlights.

```
SCRIPT >>> "TRAIN02 UNIFICADOR DATASET.py"
```



Etapa 3.2: OPTIMIZACION del DATASET

Policy Engineering (Inyección de Conocimiento Experto)

Objetivo

Reemplazar el comando crudo del humano por una etiqueta generada por reglas expertas organizadas en políticas, de forma que la DNN aprenda una política *consistente* y no los errores o inconsistencias de un jugador humano cansado, distraído o simplemente subóptimo.

Concepto clave: jerarquía de políticas

En cualquier instante, más de una política podría "querer" tomar el control de la dirección del avión (evadir una amenaza, perseguir un objetivo, buscar un recurso...). Como el avión solo puede girar en una dirección a la vez, hace falta un

mecanismo de arbitraje: una política de prioridad más alta gana y bloquea a las de menor prioridad para ese ciclo. Diseñar bien este orden de prioridades es, en sí mismo, una decisión de diseño tan importante como las reglas individuales.

Tareas específicas

Definir, para cada familia, la condición de

- **SIT** (cuándo se considera que la situación está presente)
- **COR** (cuándo, dada la situación, hace falta modificar el comando).

Implementar el arbitraje de prioridad de forma explícita (una bandera o mecanismo equivalente que impida que una política de menor prioridad sobrescriba a una de mayor prioridad ya aplicada en ese ciclo).

Documentar el orden de prioridades elegido y su justificación táctica.

Errores frecuentes en esta etapa:

Reglas demasiado estrictas: sobre-corriger eliminar variabilidad útil y le quita a la DNN la posibilidad de aprender estrategias alternativas válidas.

- **Ausencia de arbitraje de prioridades:** si dos políticas escriben el mismo comando sin un orden definido, la última que se ejecuta "gana" por accidente, no por diseño.
- **No medir SIT/COR por separado:** sin esta distinción es imposible saber después si una política está sobre-representada porque la situación es común, o porque se está corrigiendo de más.

Familia	Qué resuelve	Prioridad típica
Políticas de Seguridad	Amenazas que comprometen la supervivencia inmediata (proyectiles entrantes, límites del área de juego)	Máxima
Políticas de Recuperación	Estados físicos degradados del propio agente (pérdida de control, sobrecalentamiento, etc.) que deben corregirse sin importar qué más esté pasando	Muy alta
Políticas Tácticas	Uso de armamento u ofensiva (disparo, bloqueo de blancos)	Media-alta
Políticas Defensivas	Contramedidas ante una amenaza detectada (señuelos, interferencia)	Media-alta
Políticas de Navegación	Persecución u orientación hacia un objetivo (enemigo, punto de interés)	Media
Políticas de Gestión de Recursos	Recolección de ítems o recursos del entorno	Baja

```
SCRIPT >>> "TRAIN02_UNIFICADOR_DATASET.py"
```



Etapas 3.2: OPTIMIZACION del DATASET

Dataset Filtering y Dataset Balancing

Objetivo

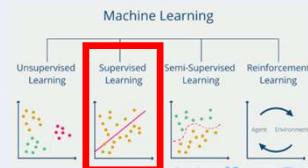
Asegurar que el dataset final contenga solo filas con información de aprendizaje real, y que ninguna política domine el entrenamiento simplemente por ser más frecuente que las demás.

4.1 Dataset Filtering:

- **Qué se descarta:** filas donde ninguna política terminó activando ningún comando (el agente "no hizo nada" desde el punto de vista de la política). Estas filas no le enseñan nada nuevo a la red sobre la relación estado → acción.
- **Qué se reporta:** cantidad de filas totales, descartadas y conservadas, para poder auditar qué porcentaje del dataset original terminó siendo útil.

Errores frecuentes:

- **Descartar filas útiles:** una fila con todos los comandos en cero podría representar una situación legítima de "espera estratégica" (por ejemplo, mantener rumbo porque nada amerita corrección). Descartarlas siempre, sin matices, puede eliminar casos válidos que la red debería poder reproducir.
- **Filtrar antes de que las políticas terminen de escribir los comandos:** si se filtra sobre el dataset crudo en vez del dataset con políticas ya aplicadas, se descartan filas por el criterio equivocado.



DATASET [SITUACIONES]														
[SENSORES]														
WHEEL	THROTTLE	STEERING	BRIDGE	WHEEL	THROTTLE	STEERING	BRIDGE	WHEEL	THROTTLE	STEERING	BRIDGE	WHEEL	THROTTLE	STEERING
STATUS	1.1	10.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
STATUS	1.1	10.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
STATUS	1.1	10.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
STATUS	1.1	10.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
STATUS	1.1	10.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
STATUS	1.1	10.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
STATUS	1.1	10.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
STATUS	1.1	10.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
STATUS	1.1	10.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
STATUS	1.1	10.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

SCRIPT >>> "TRAIN02_UNIFICADOR_DATASET.py"

Dataset Filtering y Dataset Balancing

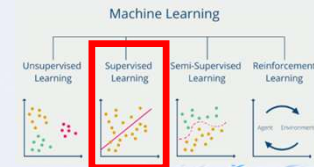
Las políticas no ocurren con la misma frecuencia: navegación y persecución son comunes; evasión de amenazas o recolección de ítems raros son escasas. Sin balanceo, la red tiende a converger hacia la política dominante y a ignorar las políticas minoritarias, que suelen ser justamente las más críticas (seguridad).

- **Oversampling:** duplicar o repetir (con o sin variaciones) las filas de las políticas minoritarias.

- **Undersampling:** reducir la cantidad de filas de la política dominante para emparejar la proporción.
- **Reweighting:** en vez de tocar el dataset, ajustar el peso de cada fila en la función de pérdida según la frecuencia de su política.
- **Balance temporal:** evitar que partidas muy largas (con muchas filas de navegación repetitiva) pesen desproporcionadamente frente a partidas cortas pero ricas en situaciones variadas.

- **Balancear antes de filtrar:** si se balancea sobre datos que todavía incluyen filas vacías, se termina sobre-representando ruido.

- **Oversampling ingenuo sin variación:** repetir filas idénticas sin ningún tipo de perturbación puede llevar a que la red memorice en vez de generalizar.
- **Balancear por política ignorando la superposición:** una misma fila puede pertenecer a más de una política simultáneamente (por el mecanismo de prioridades); balancear ingenuamente por conteo de "comando activo" puede distorsionar la proporción real de situaciones.

[illegible]

```
SCRIPT >>> "TRAIN02 UNIFICADOR DATASET.py"
```

```
SCRIPT >>> "TRAIN02 UNIFICADOR DATASET.py"
```

Action Masking

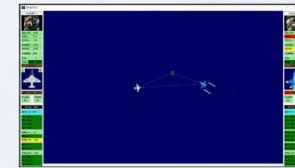
Restringir, en tiempo de inferencia, el espacio de acciones válidas según el estado actual, para que el agente no explore ni sea evaluado sobre acciones físicamente imposibles o inútiles en ese instante.

Action Masking inhibe o fuerza a probabilidad cero la elección de acciones que no tienen sentido dado el estado presente (por ejemplo, una acción ofensiva que depende de un recurso agotado, o una acción de movimiento que llevaría al agente fuera de los límites del entorno).

Ventajas

- Reduce el espacio de búsqueda efectivo.
- Acelera el entrenamiento y la convergencia.
- Evita que el agente "aprenda" a intentar acciones imposibles y sea penalizado por algo que nunca debería haber podido elegir.

- **Implementarlo dentro del pipeline de dataset en vez de en la etapa de inferencia:** mezcla responsabilidades y complica el mantenimiento del código.
- **Enmascarar de forma demasiado agresiva:** si se enmascaran acciones que en realidad sí eran viables en ciertos estados límite, se le quita al agente margen de maniobra legítimo.

[illegible]

```
SCRIPT >>> "TRAIN02_UNIFICADOR_DATASET.py"
```


ion y Dataset Validation

garando que el orden de las filas no introduzca sesgos de aprendizaje, y que existan
ra decidir si el dataset está listo para entrenamiento.

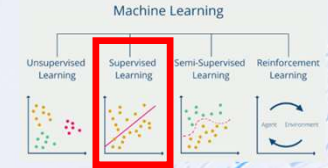
ization

- por naturaleza, ordenadas cronológicamente. Si ese orden se preserva, la red puede capturar correlaciones temporales espurias (qué viene "después de qué") en vez de la causalidad $X \rightarrow Y$. Para evitar esto, se mezclan (shuffle) las filas una vez que todas las etapas anteriores ya se aplicaron, de modo que cada fila se evalúe de forma independiente durante el entrenamiento.

2

Además del pipeline, conviene reportar un conjunto de métricas de salud del dataset:

- conservadas vs. descartadas en cada etapa.
 ración de cada política (idealmente expresada como SIT y COR por separado, como se
 tación 2).
 des aplicados por Anti-Leakage Sanitization.
 líticas de menor prioridad: ¿aparecen con suficiente frecuencia como para que la red
 representativos, o son tan raras que probablemente necesiten Balancing?
 nciona como un "gate" de calidad: si alguna política crítica de seguridad tiene muy
 una señal de que hace falta volver a alguna etapa anterior (más partidas, más
 le umbrales) antes de pasar a entrenamiento.

[illegible]

```
SCRIPT >>> "TRAIN02 UNIFICADOR DATASET.py"
```



Etapa 3.3: Reentrenar modelo con dataset optimizado.

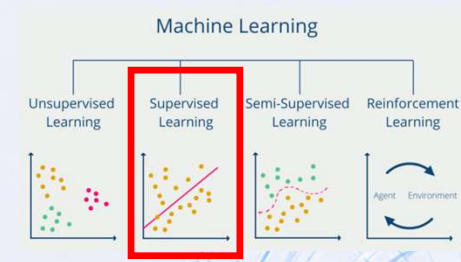
Usar el **dataset depurado y mejorado con reglas heurísticas** para volver a entrenar el modelo. El propósito es que la red neuronal aproveche **tanto la experiencia de las partidas automáticas como las correcciones impuestas por reglas humanas** para alcanzar un comportamiento más robusto y seguro en el entorno AirBattle.

Tareas específicas:

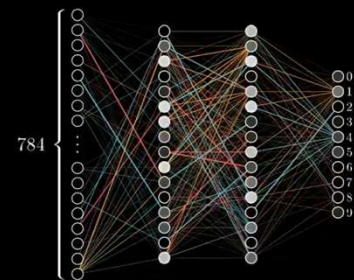
- **Cargar el nuevo dataset (Dataset_Depurado.txt):**
- **Reinicializar o cargar el modelo previo:** Cargar pesos anteriores y **seguir entrenando** para no perder el conocimiento acumulado.
- **Entrenar el modelo con el dataset modificado:** Ajustar batch_size, epochs y optimizador. Monitorear pérdida (loss) y métricas de precisión.
- **Guardar el modelo actualizado:** Exportar pesos y arquitectura en un nuevo archivo (modelo_reentrenado.h5).

Errores frecuentes en esta etapa

- **Sobrescribir el modelo previo sin respaldo:** Se pierde la comparación entre versiones (modelo inicial vs. modelo reentrenado).
- **Overfitting al dataset modificado:** El modelo memoriza las correcciones pero pierde generalización, especialmente si el dataset es pequeño.
- **Reglas mal aplicadas en 3.2 reflejadas en el modelo:** Si las reglas fueron demasiado estrictas o erróneas, el modelo aprenderá un comportamiento sesgado.
- **Olvidar probar en simulación real:** El modelo puede verse "perfecto" en métricas, pero en el juego real tomar decisiones inútiles.



Training in progress. . .



```
SCRIPT >>> "TRAIN03_AIRBATTLE_ENTRENAMIENTO.py"
```



Etapa 3.4: Evaluación de métricas del modelo

Analizar qué tanto aprendió el modelo y **cuáles entradas (inputs) tienen más impacto sobre las decisiones (outputs).**

Esto permite detectar **variables inútiles, correlaciones falsas o comportamientos inesperados** antes de lanzar otra ronda de autogeneración de partidas.

Tareas específicas:

Medir impacto de las entradas (6-ANALISIS IMPACTO.py):

- Se extraen los **pesos de la primera capa** de la red neuronal.
- Permite ver **qué inputs (ej. FUEL, STALL, ENEMY_RADAR_TRACK)** son más determinantes en la propagación hacia adelante.
- Error típico:** confundir magnitud de pesos con importancia absoluta (a veces inputs normalizados cambian la interpretación).

Analizar pesos efectivos Input-Output (8-PESOS EFECTIVOS INPUTS-OUTPUTS.py):

- Se multiplican los pesos de todas las capas para aproximar el “camino” de influencia entre cada input y cada output.
- Permite generar una **matriz de relevancia Input → Output**.
- Error típico:** interpretar correlaciones espurias como “lógica aprendida”.

Calcular correlación estadística Input-Output (7-CORRELACION INPUTS-OUTPUTS.py):

- Se miden coeficientes de correlación (Pearson o Spearman) entre las columnas de entradas y las salidas predichas.
- Diferencia con los pesos: aquí no se mide la red, sino la **relación estadística en los datos y predicciones**.
- Error típico:** usar correlación lineal en fenómenos no lineales → se pierden relaciones útiles.

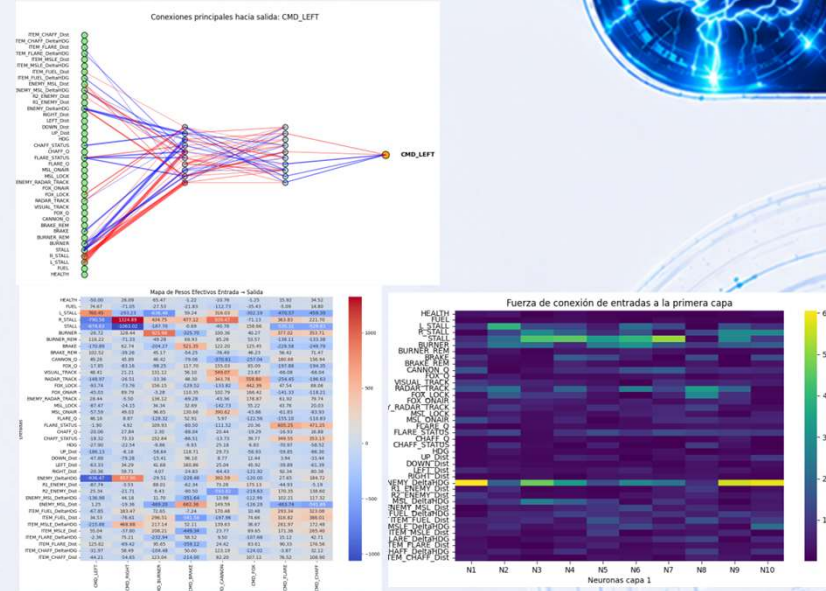
Errores frecuentes en esta etapa

Confundir métricas de entrenamiento (loss, accuracy) con análisis de pesos: Una buena accuracy no significa que el modelo esté usando los inputs correctos.

Interpretación equivocada de pesos: Pesos grandes no siempre significan más importancia (puede depender de la normalización de datos).

Omitir verificación con lógica del dominio: Se puede **aprender patrones espurios o irrelevantes** que están en los datos, aunque no tengan sentido desde el punto de vista del sistema real.

No comparar con versiones anteriores: La gracia de esta etapa es ver cómo cambió el modelo respecto al inicial y al reentrenado.



```
SCRIPT >>> "CHECK01_PESOS_EFECTIVOS_IO.py"
```

```
SCRIPT >>> "CHECK02_CORRELACION_IO.py"
```

```
SCRIPT >>> "CHECK03_GRAFICO_CONEXIONES.py"
```

```
SCRIPT >>> "CHECK04_ANALISIS_IMPACTO_INPUTS.py"
```

```
SCRIPT >>> "CHECK05_CAMINOS_EFECTIVOS.py"
```

```
SCRIPT >>> "CHECK06_CAMINOS_EFECTIVOS_IMPORT.py"
```




```
> print("Gracias !")
> print("dudas, consultas...")
> input("Presione una tecla para salir...")
```



Docente:
Gabriel Mosso



BRAINSTORM

Rep. Argentina 786, Sunchoales, Sta fe.



GabrielNMosso@GMail.com



Tel: +54 351 3733383

